DevMatch: Semantic Matching of Open-Source Issues to Contributor Skills

DBMS Experiential Learning, Level 3 Project · Department of Computer Science and Engineering, PES University · Group 6

Tejas P (SRN: PES1UG24CS497) **Tanmay T A** (SRN: PES1UG24CS492)

Source code: https://github.com/DevMatrix06/DBMS_MiniProject

1. Introduction and objectives

Finding an open-source issue you can actually do is often harder than doing it. Big repositories have hundreds of open issues, and you read them one by one until one fits. Maintainers have the opposite problem: when several people ask for the same issue, it is hard to pick the best fit. DevMatch ranks open issues for each contributor by meaning, using text embeddings stored in PostgreSQL with pgvector, and shows maintainers each applicant's skills with a match score. The problem needs strict rules (who applied, who was accepted) and fuzzy matching (what a text means) at the same time, which is why one relational database with a vector extension fits it.

- Design a normalised schema where the database itself enforces the rules (keys, CHECK, UNIQUE, cascades).
- Store embeddings next to the data and rank by similarity with pgvector, combined with keyword search (hybrid search).
- Stay correct under concurrent use with transactions, locks and triggers.
- Build a role-based Next.js app for contributors, maintainers and admins, and test it.

2. Functional requirements

Role	What the user can do
Contributor	Sign up, log in, change password, delete the account. Build a profile (bio and skills rated 1 to 5), which recomputes the profile embedding. See a paginated feed of open issues ranked by similarity, filtered by difficulty and skill. Search by keyword, meaning or both. Apply (open issues only, at most 5 pending), withdraw, and track the status of every application.
Maintainer	Create and delete repositories. Create issues, which are embedded when created or edited, and close them. Review applicants with their skills and match score, then accept one (the issue becomes claimed and the others are rejected) or reject.
Admin	Platform statistics read from SQL views: repositories, skill demand against supply, open issues, users and applications.
Everyone	bcrypt passwords with a JWT in an httpOnly cookie; role check on every API; secrets only in `.env`; validation in the API and again in the database; login rate limit; audit log of issue status changes; 40 automated tests.

3. ER diagram

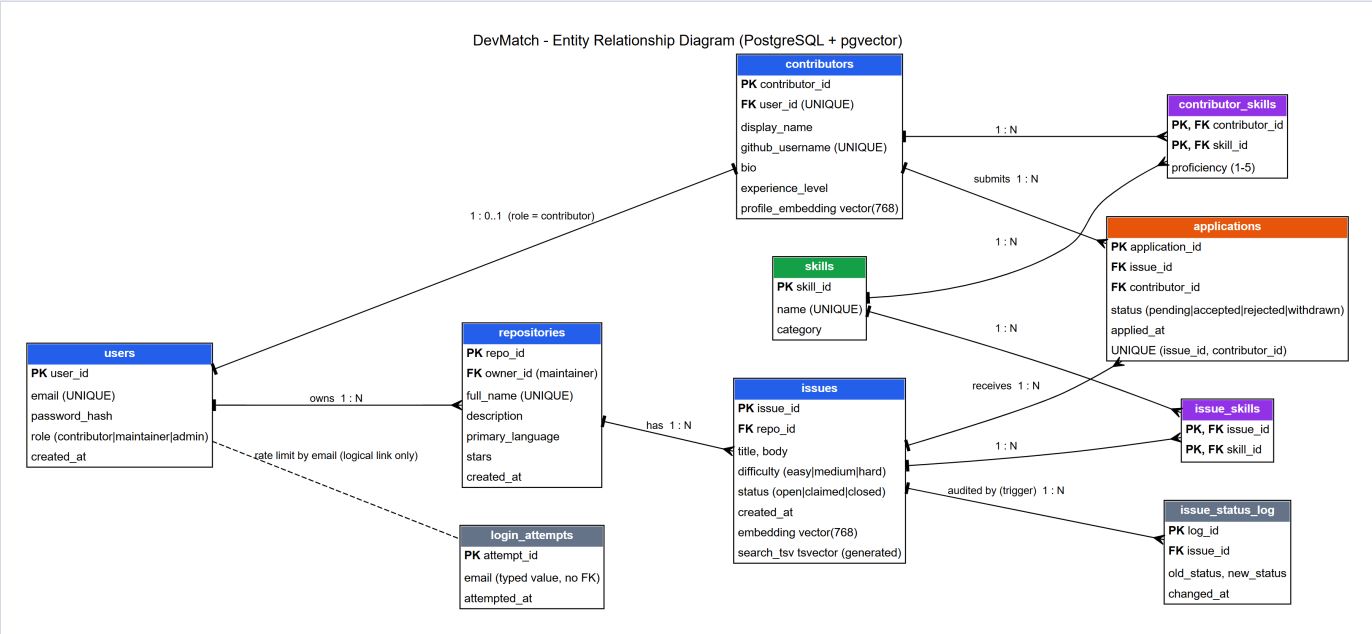


Figure 1. ER diagram: ten tables. Blue are the main entities, green skills, purple the junction tables, orange applications, grey logging and security.

A user has at most one contributor profile, and a maintainer owns many repositories. A repository has many issues. Issues and contributors both link to skills through junction tables, and an application links one contributor to one issue (at most once). A trigger fills `issue_status_log`. `login_attempts` has no foreign key on purpose: it stores the typed email, so guesses for unknown accounts are limited too.

4. Relational schema

```
users(user_id PK, email UQ, password_hash, role, created_at)
contributors(contributor_id PK, user_id UQ FK->users, display_name, github_username UQ,
             bio, experience_level, profile_embedding)
skills(skill_id PK, name UQ, category)
contributor_skills(contributor_id FK->contributors, skill_id FK->skills, proficiency,
                   PK(contributor_id, skill_id))
repositories(repo_id PK, owner_id FK->users, full_name UQ, description, primary_language,
             stars, created_at)
issues(issue_id PK, repo_id FK->repositories, title, body, difficulty, status, created_at,
       embedding, search_tsv)
issue_skills(issue_id FK->issues, skill_id FK->skills, PK(issue_id, skill_id))
applications(application_id PK, issue_id FK->issues, contributor_id FK->contributors,
             status, applied_at, UQ(issue_id, contributor_id))
issue_status_log(log_id PK, issue_id FK->issues, old_status, new_status, changed_at)
login_attempts(attempt_id PK, email, attempted_at)
```

All tables are in third normal form: skill names live once in `skills`, many-to-many links are junction tables, and every other column depends only on its own table's key. Every foreign key is `ON DELETE CASCADE`.

5. Database choice and justification

- **PostgreSQL for the structured data.** Users, repositories, issues and applications are linked by foreign keys, and the rules that matter are integrity rules. A relational database enforces them and gives us transactions, row locks, triggers, views and joins.
- **pgvector in the same database for embeddings.** Real queries mix ordinary filters (open, easy, needs Python) with similarity ranking. With the vector in the same row, one query filters, joins and ranks, in one transaction. A separate vector database would mean syncing two systems and merging results in code.
- **Not a document database.** Every entity has fixed attributes and many-to-many links, which is what tables are for. Unstructured text is handled by embeddings and a full-text index.

6. Data dictionary

Table	Columns and constraints
`users`	user_id PK; email UNIQUE, CHECK valid format; password_hash; role CHECK contributor / maintainer / admin; created_at
`contributors`	contributor_id PK; user_id UNIQUE FK; display_name 1 to 100 chars; github_username UNIQUE; bio; experience_level CHECK beginner / intermediate / advanced; profile_embedding vector(768)
`skills`	skill_id PK; name UNIQUE; category CHECK language / framework / database / devops / domain / other
`contributor_skills`	PK (contributor_id FK, skill_id FK); proficiency CHECK 1 to 5
`repositories`	repo_id PK; owner_id FK to users; full_name UNIQUE; description; primary_language; stars CHECK >= 0; created_at
`issues`	issue_id PK; repo_id FK; title CHECK 5 to 300 chars; body; difficulty CHECK easy / medium / hard; status CHECK open / claimed / closed; created_at; embedding vector(768); search_tsv tsvector (generated)
`issue_skills`	PK (issue_id FK, skill_id FK)
`applications`	application_id PK; issue_id FK; contributor_id FK; status CHECK pending / accepted / rejected / withdrawn; applied_at; UNIQUE (issue_id, contributor_id)
`issue_status_log`	log_id PK; issue_id FK; old_status; new_status; changed_at (written by a trigger)
`login_attempts`	attempt_id PK; email (as typed, no FK); attempted_at

Indexes: HNSW (cosine) on both embedding columns, GIN on `search_tsv`, B-tree on foreign keys, on `issues(status, difficulty)` and on `issue_skills(skill_id)`. **Views:** `v_open_issues`, `v_skill_demand`, `v_repo_stats`.

7. Implementation

Plain SQL files build the whole database; Docker runs them the first time the container starts. Rules that must never be broken live in the database, so they hold for any code or manual SQL:

- `trg_issue_status_log` records every change of an issue's status.
- `trg_application_rules` allows an application only on an open issue and at most 5 pending per contributor. It takes an advisory lock per contributor so simultaneous requests cannot slip past the limit.
- `trg_reject_pending_on_close` rejects an issue's pending applications when it is closed.
- `match_issues()` orders open issues by cosine distance to a contributor's profile. `hybrid_search()` merges the top 30 keyword and top 30 semantic results with Reciprocal Rank Fusion (`1/(60 + rank)` from each list), with the difficulty and skill filters applied before ranking.

Accepting an applicant is one transaction that locks the issue row, so two maintainers clicking together cannot both win:

```
BEGIN;
SELECT i.status FROM issues i JOIN repositories r ON r.repo_id = i.repo_id
  WHERE i.issue_id = $1 AND r.owner_id = $2 FOR UPDATE;   -- lock, and prove ownership
UPDATE applications SET status = 'accepted' WHERE application_id = $3;
UPDATE applications SET status = 'rejected'
  WHERE issue_id = $1 AND application_id <> $3 AND status = 'pending';
UPDATE issues SET status = 'claimed' WHERE issue_id = $1;  -- trigger logs open -> claimed
COMMIT;
```

Vector search. Issues are embedded from title, body and skills; profiles from bio and skills; queries from the typed text. We use `gemini-embedding-001`, reduced to 768 numbers and scaled to unit length, stored in `vector(768)` columns and compared with cosine distance, with an HNSW index. Searching "screen reader support" finds "Dropdown menu is not keyboard accessible" by meaning, while keyword search finds nothing.

8. Important queries

Query (db/05_queries.sql)	Concepts	What it answers
Q2 contributors and average proficiency per skill	JOIN, GROUP BY, AVG	How strong the community is per skill
Q3 skills with more open issues than contributors	GROUP BY, HAVING	Where we lack contributors
Q4 contributors who never applied	NOT EXISTS (correlated subquery)	Who to encourage
Q5 repository with the highest share of hard issues	Subquery in HAVING	Most demanding project
Q7 open issues nobody applied to	LEFT JOIN ... IS NULL	Issues needing attention
Q9 top 5 matches for a contributor	Vector function and JOIN	The matching feature in SQL

```
-- Q3 [GROUP BY, HAVING] Skills with MORE open issues than contributors (a talent shortage).
--    HAVING filters after aggregation, which WHERE cannot do.
SELECT s.name,
       COUNT(DISTINCT isk.issue_id)        AS open_issues,
       COUNT(DISTINCT cs.contributor_id)   AS contributors
FROM skills s
JOIN issue_skills isk            ON isk.skill_id = s.skill_id
JOIN issues i                    ON i.issue_id = isk.issue_id AND i.status = 'open'
LEFT JOIN contributor_skills cs  ON cs.skill_id = s.skill_id
GROUP BY s.skill_id
HAVING COUNT(DISTINCT isk.issue_id) > COUNT(DISTINCT cs.contributor_id)
ORDER BY open_issues DESC, s.name;
```

9. System architecture

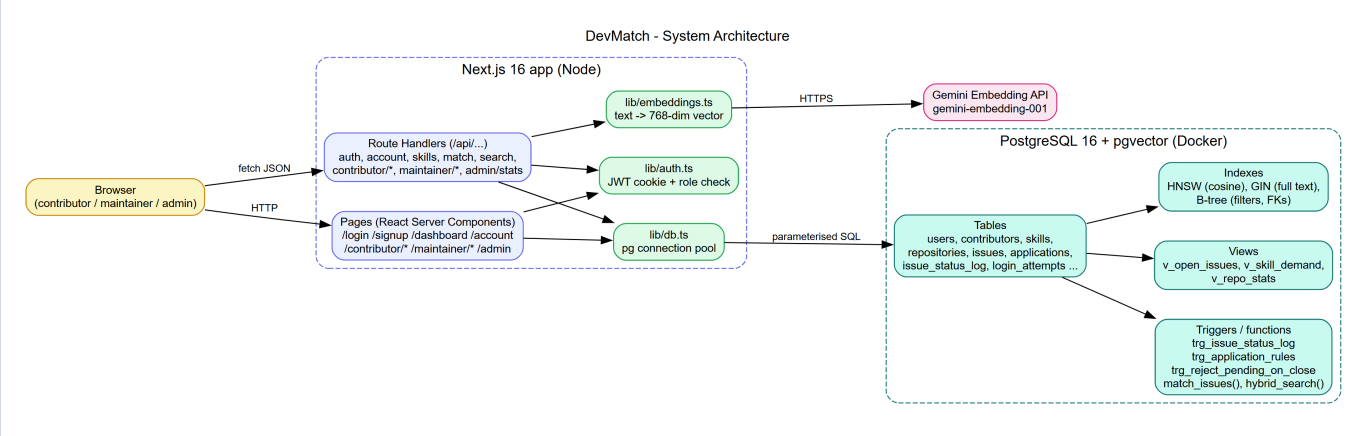


Figure 2. Architecture: the browser calls Next.js pages and API route handlers; the app uses one PostgreSQL connection pool and the Gemini API for embeddings.

10. Application screenshots

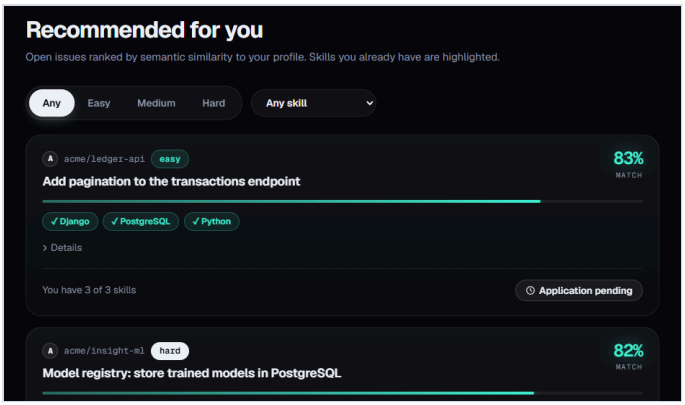


Figure 3. Contributor feed: issues ranked by match percentage; ticked skills are ones she already has.

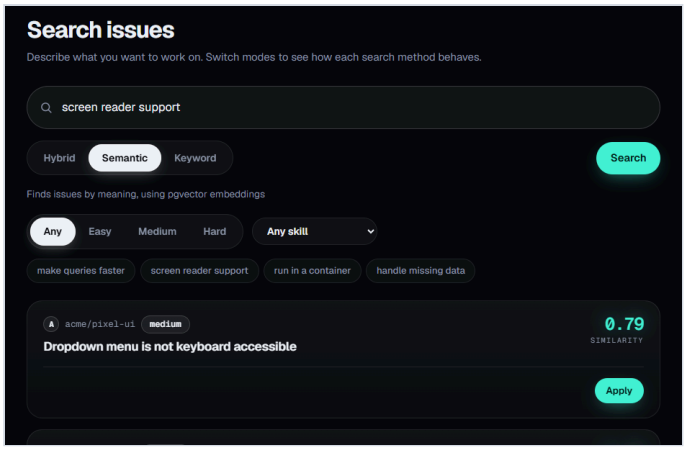


Figure 4. Semantic search finds the keyboard issue for "screen reader support"; keyword search finds nothing.

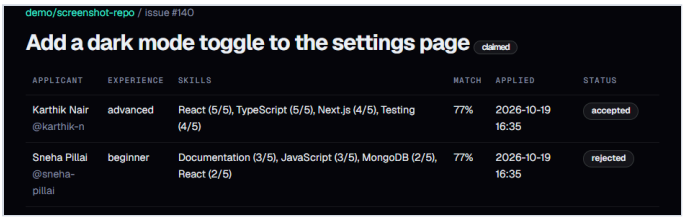


Figure 5. Review screen after accepting one applicant: the other is rejected, the issue is claimed.

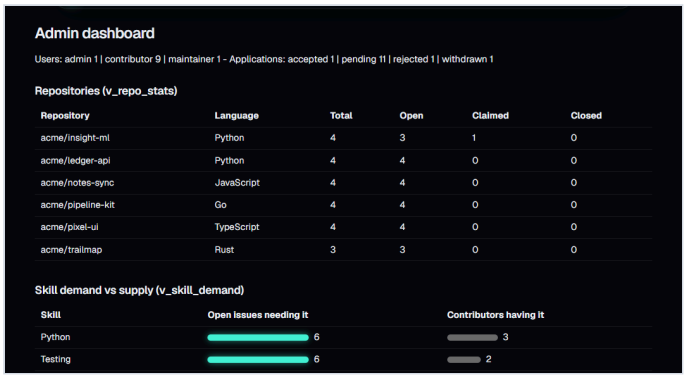


Figure 6. Admin dashboard, read from the three SQL views.

11. Testing results

40 integration tests call the running app over HTTP and check the real database. Each run creates and deletes its own data. **All 40 pass**, including two race-condition tests: six simultaneous applications from one contributor (exactly five succeed) and two simultaneous accepts (exactly one wins).

Group	Tests	Covers
Authentication	A1 to A8	Login, duplicate and malformed email, weak password, admin role blocked at signup, generic error messages, logout, login rate limit
Role-based access	R1 to R4	Contributor, maintainer and admin boundaries on every API
Profile, search, filters	P1, P2, S1, S2, F1, F2, H1	Validation, embedding stored, ranked feed, semantic versus keyword, hybrid fusion, difficulty and skill filters
Applications	AP1 to AP7, CL1	Apply, withdraw, re-apply, 5-pending limit (including a simultaneous-request race), close rule
Maintainer flow	M1 to M3	Accept transaction, concurrent accepts (exactly one wins), embedded issue creation
Account	PW1, DEL1 to DEL4	Password change, cascading deletion of accounts and repositories, admin protection
Database integrity	D1 to D5	CHECK, FK, UNIQUE, trigger, views and functions, cascade

Testing also found real problems, now fixed: applying to a missing issue gave a misleading error; a fresh database failed to start because Docker also ran our demo scripts; and a teammate's older database had no `login_attempts` table, so sign-in failed (we added a clear message, a schema check in `/api/health` and `npm run db:update`).

12. What we learned and conclusion

Concurrency was the part we underestimated. Our first version checked the five-application limit only in the API, and it looked fine until we sent requests at the same moment. Moving the rule into a trigger with a lock, and writing a test that really fires six requests together, taught us more than any example. We also learned that embedding scores are fuzzy (ours sit between 75 and 83 percent, so the order matters more than the number), and that two laptops mean two databases, so schema changes need a migration path from day one.

DevMatch shows that one relational database can hold both the strict rules of an issue tracker and the fuzzy matching that makes it useful: constraints, transactions, locks and triggers keep the data correct, and pgvector ranks by meaning inside the same SQL engine. Next we would add notifications, a reason for rejection, and an admin screen for managing users.

To run it: copy `.env.example` to `.env`, then `docker compose up -d`, `npm install`, `npm run seed` and `npm run dev` (http://localhost:3000). Logins such as `asha@devmatch.dev`, `maintainer@devmatch.dev` and `admin@devmatch.dev` use the password `password123`.